

Hands-On Lab: Async and Multithreaded Programming in C#

Overview

In this lab, you'll learn how to use `async` and `await` for asynchronous programming and utilize concurrent collections from the `System.Collections.Concurrent` namespace. You will build a simple web scraper that downloads content from multiple websites in parallel and processes the data concurrently.

Prerequisites

- Basic understanding of C# and .NET.
 - Familiarity with `Task`, `async`, and `await`.
 - Visual Studio or a similar IDE.
-

Objectives

By the end of this lab, you will:

1. Understand how to use `Task` and `async/await` for asynchronous operations.
 2. Work with `System.Collections.Concurrent` to manage multithreaded data processing.
 3. Learn to handle exceptions in asynchronous code.
-

Step 1: Setup the Project

1. Open Visual Studio and create a new **Console App** project.
 2. Name the project `AsyncMultithreadedLab`.
-

Step 2: Build the Web Scraper

1. Add the following NuGet package for HTTP requests:

```
dotnet add package System.Net.Http
```

2. Create a new class named `WebScraper` with the following code:

```
using System.Net.Http;
```

```

using System.Threading.Tasks;

public class WebScraper
{
    private static readonly HttpClient _httpClient = new();

    public async Task<string> FetchWebsiteContentAsync(string url)
    {
        try
        {
            Console.WriteLine($"Fetching content from {url}...");
            string content = await _httpClient.GetStringAsync(url);
            Console.WriteLine($"Completed fetching content from {url}");
            return content;
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error fetching {url}: {ex.Message}");
            return string.Empty;
        }
    }
}

```

Step 3: Use `Task.WhenAll` for Parallel Fetching

1. In the `Program.cs` file, add the following:

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        WebScraper scraper = new();
        var urls = new List<string>
        {
            "https://example.com",
            "https://www.google.com",
            "https://www.microsoft.com"
        };

        Console.WriteLine("Starting to fetch website content...");
        var fetchTasks = new List<Task<string>>();
    }
}

```

```

        foreach (var url in urls)
        {
            fetchTasks.Add(scraper.FetchWebsiteContentAsync(url));
        }

        var results = await Task.WhenAll(fetchTasks);

        Console.WriteLine("All content fetched. Processing...");
        foreach (var content in results)
        {
            Console.WriteLine($"Content length: {content.Length}");
        }
    }
}

```

2. Run the program and observe the output.

Step 4: Use Concurrent Collections

To process the content concurrently, use a `ConcurrentQueue`.

1. Modify the `Program.cs` file:

```

using System.Collections.Concurrent;

class Program
{
    static async Task Main(string[] args)
    {
        WebScraper scraper = new();
        var urls = new List<string>
        {
            "https://example.com",
            "https://www.google.com",
            "https://www.microsoft.com"
        };

        var contentQueue = new ConcurrentQueue<string>();
        var fetchTasks = new List<Task>();

        foreach (var url in urls)
        {
            fetchTasks.Add(Task.Run(async () =>
            {
                var content = await scraper.FetchWebsiteContentAsync(url);
                contentQueue.Enqueue(content);
            }));
        }
    }
}

```

```

    }

    await Task.WhenAll(fetchTasks);

    Console.WriteLine("Processing content in the queue...");
    Parallel.ForEach(contentQueue, content =>
    {
        Console.WriteLine($"Processed content of length: {content.Length}");
    });
}
}

```

Step 5: Handle Exceptions

Modify the fetch loop to handle exceptions during parallel processing.

1. Update the `foreach` loop:

```

foreach (var url in urls)
{
    fetchTasks.Add(Task.Run(async () =>
    {
        try
        {
            var content = await scraper.FetchWebsiteContentAsync(url);
            contentQueue.Enqueue(content);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error processing {url}: {ex.Message}");
        }
    }));
}
}

```

Step 6: Experiment with Scaling

1. Add more URLs to the `urls` list.
2. Observe how the program behaves with different numbers of URLs.

Summary

In this lab, you:

1. Built an asynchronous web scraper using `async` and `await`.

2. Processed data concurrently using `ConcurrentQueue` and `Parallel.ForEach`.
3. Handled exceptions in a multithreaded environment.